

정보과학 프로젝트

프로젝트 계획서

이 계획서는 내 웹 서비스의 설계도입니다. 계획은 진행하면서 바뀔 수 있고, 바뀐 과정도 성장의 기록이 됩니다.
회색 글씨는 작성 안내와 예시입니다. 안내를 참고해 나의 내용으로 채워 보세요.

학번	2404	이름	강경빈
프로젝트명	오늘 얼마? - 다음 입금일까지 버티는 학생 지출 관리 서비스		
사용 AI 도구	☐ ChatGPT(Codex) ☒ Claude(Claude Code)		

■ 1. 어떤 서비스인가요? - 서비스 개요

돕고 싶은 사람 (사용자)	아르바이트비와 용돈처럼 들어오는 날짜와 금액이 매번 달라지는 수입으로 한 달을 생활하는 대학생을 돕고 싶은 사람으로 설정했다.. 고정 수입이 있는 직장인과 달리 배달음식, 카페, 친구들과의 약속처럼 즐거움을 위한 지출 비중이 크고, 그러다 월말이 되면 잔액이 부족해 밥을 굶거나 버스를 타지 못하는 경험을 반복하는 10대 후반~20대 초반 학생을 대상으로 한다. 가계부를 써야 한다는 것은 알지만 매번 기록하는 일이 번거로워 며칠 만에 포기해 본 적이 있는 사람에게도 도움이 될 것이다.
이 사람이 겪는 문제·불편	기존 가계부 앱은 매달 같은 날 같은 금액이 들어오는 직장인의 월급을 기준으로 예산을 짜기 때문에, 입금일과 금액이 매번 달라지는 대학생에게는 잘 맞지 않는다. 또 대부분 '이번 달에 얼마를 썼는지'를 알려 줄 뿐, '지금부터 다음 알바비가 들어오는 날까지 하루에 얼마씩 써도 되는지'는 알려 주지 않는다. 그래서 돈이 들어온 직후에는 마음 놓고 쓰다가 월말에 교통비와 밥값이 모자라는 일이 반복된다. 지출을 하나하나 입력하는 과정도 번거로워 기록 자체를 중간에 포기하게 되는 것도 문제다.
서비스 한 줄 소개	대학생이 현재 잔액과 다음 수입 예정일, 그리고 그날그날의 지출을 입력하면, 다음 입금일까지 남은 일수로 나눈 '오늘/이번주 쓸 수 있는 안전 금액'과 카테고리별 남은 예산, 이대로 쓰면 잔액이 0원이 되는 예상 날짜 그래프를 한 화면에 보여 주는 서비스

■ 2. IPO 시스템 설계

우리가 만드는 서비스는 반드시 입력(Input) → 처리(Process) → 출력(Output) 구조를 갖습니다. 각 칸의 예시를 참고해 나의 서비스로 채우세요.

입력 (Input)	① 현재 통장 잔액과 가진 현금 ② 다음 수입의 종류(알바비·용돈·과외비)와 입금 예정일, 예상 금액 ③ 통신비·구독료·교통 정기권 등 매달 빠져나가는 고정지출의 항목명과 금액, 결제일 ④ 지출을 기록할 때의 금액과 카테고리(식비·배달·카페·유흥/모임·교통·기타), 날짜, 간단한 메모 ⑤ '학식 5,000원', '버스 1,500원'처럼 자주 쓰는 항목을 미리 등록해 두는 원터치 빠른 입력 버튼 ⑥ 다음 주 생일 모임처럼 이미 예정된 지출의 날짜와 예상 금액
처리 (Process)	① 오늘부터 다음 입금 예정일까지 남은 일수를 센다. ② 현재 잔액에서 그 기간에 빠져나갈 고정지출과 사용자가 미리 등록한 예정 지출을 먼저 빼서 '실제로 쓸 수 있는 돈'을 구한다. ③ 이 금액을 남은 일수로 나누어 '오늘의 안전 지출 금액'을 계산한다. ④ 카테고리별로 정해 둔 예산에서 이미 쓴 금액을 빼 남은 예산과 소진율(%)을 구한다. ⑤ 최근 7일간의 하루 평균 지출 속도를 구하고, 이 속도가 유지될 경우 잔액이 0원이 되는 날짜를 예측한다. ⑥ 지출이 새로 기록될 때마다 위 계산을 전부 다시 수행한다.
출력 (Output)	계산 결과를 잔소리나 경고 없이 숫자와 그래프로만 보여 준다. ① 메인 화면 한가운데에 '오늘 12,400원까지 괜찮아요'처럼 오늘의 안전 지출 금액을 가장 크게 표시한다. ② 그 아래에 식비·배달·카페·유흥/모임·교통 등 카테고리별 남은 예산을 가로 막대 그래프로 나란히 놓아, 어느 항목에 여유가 있고 어느 항목이 빠듯한지 한눈에 비교되게 한다. ③ 통계 화면에서는 오늘부터 다음 입금일까지의 잔액 변화를 꺾은선 그래프로 그리고, 잔액이 0원이 되는 예상 날짜에 표시를 찍어 언제쯤 부족해지는지 보여 준다. ④ 지출을 기록하면 세 가지 결과가 즉시 다시 계산되어 화면에 반영된다.
저장할 데이터 (2단계·Supabase)	① 사용자 정보(로그인 계정, 닉네임) ② 수입 기록(종류, 금액, 입금 예정일, 실제 입금 여부) ③ 지출 기록(금액, 카테고리, 날짜, 메모) ④ 카테고리별 예산 설정값 ⑤ 통신비·구독료 같은 고정지출 목록 ⑥ 원터치 빠른 입력 버튼으로 등록해 둔 항목(이름, 금액, 카테고리)이다. 금액은 민감한 개인 정보이므로 Supabase의 RLS(행 수준 보안) 기능을 켜서 로그인한 본인의 데이터만 저장·조회되도록 설정한다.

■ 3. 화면 구성 스케치

핵심 화면 3개 이상을 그려 보세요. 입력 칸·버튼·결과가 어디에 보이는지 드러나면 충분합니다. 이 스케치는 AI에게 화면을 설명할 때 좋은 자료가 됩니다.



■ 4. 차시별 개발 계획

전체 흐름: ① 잘 동작하는 웹 프론트 만들기 → ② Supabase DB를 붙여 서버 기능 추가. 회색 예시를 참고해 나의 계획으로 고쳐 쓰세요.

차시	단계	이번 차시 목표
1	준비	GitHub 저장소를 만들고 Claude Code를 연결한다. 주제와 핵심 화면 3개(메인·지출 입력·통계)를 확정하고 지출 카테고리 목록을 정한다.
2	프론트 개발	메인 화면의 뼈대를 만든다. 상단 잔액, 중앙의 '오늘의 안전 지출 금액', 하단 버튼 자리를 배치하고 임시 숫자가 화면에 보이는 것까지 확인한다.
3	프론트 개발	지출 입력 화면을 완성한다. 금액·카테고리·날짜를 입력하고 저장을 누르면 목록에 한 줄이 추가되는 것을 확인하고, 원터치 빠른 입력 버튼 3개도 만든다.
4	프론트 개발	안전 지출 금액 계산 기능을 구현한다. 잔액·다음 입금일·고정지출을 넣으면 남은 일수로 나눈 하루 금액이 메인 화면에 실제로 표시되게 한다.
5	프론트 개발	카테고리별 남은 예산 막대 그래프와 잔액 예측 꺾은선 그래프를 완성하고, Vercel에 첫 배포해 휴대폰 브라우저에서 접속되는지 확인한다.
6	서버·DB 추가	Supabase 프로젝트를 만들고 사용자·수입·지출·예산 테이블을 설계한다. 앱과 연결해 로그인과 로그아웃이 동작하는 것까지 확인한다.
7	서버·DB 추가	지출을 입력하면 Supabase에 실제로 저장되게 만든다. RLS를 켜서 로그인한 본인의 데이터만 저장·조회되는지 확인한다.
8	서버·DB 추가	저장된 데이터를 불러와 안전 지출 금액과 그래프를 다시 계산해 보여 준다. 새로고침해도 기록이 남아 있는지 확인하고 최종 배포한다.

■ 5. 예상되는 어려움과 나의 대비

막힐 지점을 미리 예상해 두면, 실제로 겪었을 때의 성장이 잘 보입니다.

예상되는 어려움	나의 대비 방법
웹 개발이 거의 처음이라 화면을 만드는 기본 문법과 파일 구조 자체를 몰라 첫 화면부터 막힐 것 같다.	Claude Code에 한 번에 한 화면씩만 요청하고, 만들어진 코드를 한 줄씩 설명해 달라고 해서 이해한 뒤 다음 단계로 넘어간다. 차시마다 동작하는 상태로 커밋해 둔다.
안전 지출 금액을 계산할 때 남은 일수나 고정지출 처리를 잘못해 결과 숫자가 이상하게 나올 수 있다.	잔액 30만 원, 남은 일수 10일처럼 직접 손으로 계산한 예시 값을 미리 정해 두고, 화면에 나온 숫자와 비교해 검산한 뒤 다음 기능으로 넘어간다.

■ 6. 이 프로젝트로 성장하고 싶은 것 (다짐)

결과 보고서 마지막에서 이 다짐과 실제 나의 변화를 비교하게 됩니다.(현재 나의 상태(생성형 AI를 통한 웹 개발 경험, 깃허브 사용 경험 등등)) 구체적으로 쓸수록 비교가 쉽습니다.

지식·기술 면	지금까지 GitHub는 다른 사람이 올려 둔 코드를 내려받는 용도로만 써 봤고, 직접 웹 서비스를 만들어 본 경험은 거의 없다. 이번 프로젝트에서는 먼저 입력(Input)·처리(Process)·출력(Output) 구조를 내 서비스에 직접 대응시켜 보고, 사용자가 입력한 값이 어떤 계산을 거쳐 화면의 숫자로 바뀌는지 그 흐름을 코드 단위로 설명할 수 있게 되고 싶다. 특히 '오늘의 안전 지출 금액'처럼 내가 직접 세운 계산 규칙을 함수로 옮기고, 값이 틀렸을 때 어느 단계가 잘못됐는지 찾아내는 경험을 해 보고 싶다. 또 Supabase에 데이터를 저장하고 다시 불러오는 과정을 겪으면서, 화면에만 있던 정보가 어떻게 서버에 남아 새로고침 후에도 유지되는지 원리를 이해하고 싶다. 학기가 끝날 때는 GitHub에 커밋 기록을 쌓고 Vercel에 배포한 주소를 친구에게 보내 실제로 써 보게 하는 것이 목표다.
태도·습관 면	웹 개발이 처음이다 보니 오류 메시지가 뜨면 겁부터 나서 그대로 복사해 AI에게 던지고 답만 받아 붙여 넣게 될 것 같다. 하지만 그렇게 하면 코드는 늘어나도 내가 이해한 것은 늘지 않는다는 것을 알고 있다. 그래서 이번 학기에는 오류가 났을 때 최소 5분은 스스로 오류 메시지를 읽고, 어느 파일 몇 번째 줄에서 무엇이 문제인지 먼저 짐작해 본 뒤에 질문하는 습관을 만들고 싶다. 또 AI가 만들어 준 코드를 그냥 넘기지 않고 '이 부분이 왜 필요한지'를 되물어 확인한 다음 저장하려고 한다. 그리고 매 차시가 끝날 때마다 그날 무엇을 했고 어디서 막혔는지 한 줄이라도 기록해 두어, 학기 말 결과 보고서에서 내가 처음과 어떻게 달라졌는지 스스로 비교할 수 있게 남겨 두고 싶다.